

adc_cpp : architecture d'un solveur AMR / MPI / GPU

Script de présentation et questions anticipées

Romain Despoullains — LPP

Format. 13 diapositives, viser ~12 min de parole, ~1 min par diapo (la timeline « construire son solveur » un peu plus). Garder le ton : on présente une *conception*, pas un catalogue de fonctionnalités. Le fil : une équation $\rightarrow$ deux contrats $\rightarrow$ un coupleur $\rightarrow$ on construit un solveur en 5 étapes.

Script diapo par diapo

[Titre] Architecture d'un solveur AMR / MPI / GPU

Ce que je dis : « Je présente `adc_cpp`, un solveur C++23 pour des systèmes couplés hyperbolique–elliptique, du type qu'on rencontre en physique des plasmas. L'angle n'est pas la physique d'un cas précis mais la *conception* : comment on écrit un code qui tourne en série, en OpenMP, en MPI et sur GPU sans réécrire les opérateurs. »

Message clé : On parle d'architecture logicielle au service de la physique.

[1/11] Le problème

Ce que je dis : « Le problème générique : une partie hyperbolique, le champ conservé U qui est transporté, et une partie elliptique, un potentiel ϕ solution d'une équation de Poisson $\Delta\phi = f(U)$. Les deux sont couplées à *chaque pas* par la variable auxiliaire $\text{aux} = (\phi, \nabla\phi)$: le potentiel entre dans le flux de la partie hyperbolique. Le diocotron, l'Euler–Poisson auto-gravitant rentrent dans ce moule. »

Message clé : Toute la suite découle de ce découpage hyperbolique / elliptique / couplage.

[2/11] Architecture du solveur

Ce que je dis : « Cinq couches, une responsabilité chacune. **PhysicalModel** : les lois locales (flux, sources, fermetures), et rien d'autre — c'est device-callable. **NumericalMethod** : reconstruction, flux numérique, opérateur elliptique, CL. **DataLayout** : les conteneurs, MultiFab / BoxArray / Geometry / hiérarchie AMR. **ExecutionBackend** : les *seams*, les coutures où le parallélisme est isolé — `for_each_cell`, la comm MPI, l'allocateur. Au-dessus, **TimeIntegrator** compose les opérateurs (RK, IMEX, splitting) sans connaître leur implémentation. Le point clé : on n'écrit de la physique qu'au sommet, et le backend ne voit que des vues minimales (une Box, un Array4, un scalaire, un rang). Il ne touche jamais à BoxArray. C'est ça qui rend le passage GPU / MPI mécanique. »

Message clé : Les couches sont étanches ; le parallélisme vit dans une seule couche, les seams.

[3/11] PhysicalModel (le concept)

Ce que je dis : « Un modèle, c'est un *type* qui fournit quatre fonctions locales : `flux`, `max_wave_speed` (qui donne le pas de temps CFL), `source`, et `elliptic_rhs` (le terme source de Poisson, le couplage). C'est un *concept* C++20 : le compilateur vérifie le contrat. Pas d'héritage, pas de fonction virtuelle. Tout type qui remplit ces quatre signatures est un modèle valide, et le code est entièrement inliné. »

Message clé : Le contrat est vérifié à la compilation, pas par une hiérarchie de classes.

[4/11] Exemple : le diocotron

Ce que je dis : « Voici un modèle concret qui remplit ce contrat : le diocotron. Une seule variable, la densité électronique n_e . Le flux est le transport par la dérive $E \times B$: $v = (E \times B) / |B|^2$, et $F = n_e v$.

Le terme elliptique, c'est la densité moins un fond ionique neutralisant. Toutes les fonctions sont marquées `ADC_HD` : la même ligne de code compile sur CPU et sur GPU. C'est exactement les quatre fonctions du concept, écrites pour un cas physique. »

Message clé : Le concept de la diapo précédente, instancié sur une vraie physique.

[5/11] EllipticSolver (le concept)

Ce que je dis : « Le jumeau elliptique. Là où `PhysicalModel` décrit la physique locale, `EllipticSolver` décrit « tout ce qui sait résoudre $\Delta\phi = f$ ». Trois méthodes : `rhs()` où on écrit le second membre f , `solve()` qui résout, `phi()` où on lit la solution. Le `Coupler<Model, Elliptic>` est le seul à connaître les deux contrats : il pousse f , résout, relit ϕ , calcule $\nabla\phi$, le range dans `aux`, et le redonne au flux. Trois backends remplissent ce concept et sont interchangeables : multigrille géométrique, FFT spectral, FFT distribué MPI. »

Message clé : Deux contrats indépendants, reliés par un seul coupleur via $\text{aux} = (\phi, \nabla\phi)$.

[6/11] Construire son solveur avec adc

Ce que je dis : « Maintenant, la partie pratique : comment un utilisateur construit son solveur. En cinq étapes — définir le modèle, le faire tourner sur grille uniforme, choisir le schéma, passer en AMR, choisir le backend. À droite, le diocotron tournant en AMR : on voit les patches raffinés (les rectangles) suivre la zone de fort gradient. »

Message clé : De zéro à un solveur AMR/GPU en cinq gestes, sans toucher au moteur.

[7/11] Étape 1 : définir son modèle

Ce que je dis : « On reprend tout depuis le début avec un modèle d'advection scalaire à vitesse (a, b) . On remplit les quatre fonctions, et la dernière ligne, le `static_assert`, demande au compilateur de vérifier que le type satisfait bien `PhysicalModel`. Si une signature manque, ça ne compile pas. »

Message clé : Écrire un modèle = remplir un `struct` et laisser le compilateur valider.

[8/11] Étape 2 : grille uniforme

Ce que je dis : « On instancie un `Coupler<Diocotron, GeometricMG>` avec une géométrie et des conditions limites. La boucle en temps appelle `advance`. Le coupleur ferme la boucle $\phi \rightarrow \text{aux} \rightarrow \text{avance}$, automatiquement. Pour un modèle purement hyperbolique sans Poisson, on appellerait directement `advance_ssprk2`. »

Message clé : Le couplage Poisson-transport est encapsulé ; l'utilisateur n'écrit que la boucle.

[9/11] Étape 3 : choisir le schéma

Ce que je dis : « Le flux et la reconstruction sont des *policies* template, passées en paramètre, pas des `if` dans une boucle. `advance<VanLeer>` donne du MUSCL van Leer. On a Rusanov, HLL, HLLC pour le flux ; NoSlope, Minmod, van Leer, MC, WENO5-Z pour la reconstruction. Changer de schéma ne touche jamais le modèle, et comme c'est résolu à la compilation, il n'y a aucun surcoût de branche. »

Message clé : Le schéma numérique est un paramètre, orthogonal à la physique.

[10/11] Étape 4 : passer en AMR

Ce que je dis : « Même interface, un coupleur de plus : `AmrCouplerMP`. On déclare les niveaux (grossier + patches fins), on donne un critère de raffinement (ici : densité au-dessus d'un seuil), et la boucle appelle `regrid` périodiquement puis `step`. Le moteur gère la hiérarchie, le clustering Berger-Rigoutsos, le sous-cyclage et le reflux conservatif. L'utilisateur ne fournit que le critère. »

Message clé : L'AMR est un changement de coupleur, pas une réécriture du solveur.

(Note : le footer de cette diapo dit « regrid, reux » — coquille pour « reflux ». Dire « reflux » à l'oral.)

[11/11] Étape 5 : choisir le backend

Ce que je dis : « Et le code source ne change pas. Le backend se choisit une seule fois, à `cmake` : un drapeau pour OpenMP, un pour MPI, un pour Kokkos/GPU. Le `seam for_each_cell` se compile en boucle série, en `#pragma omp`, ou en `Kokkos::parallel_for` selon le drapeau. Passer en GPU, c'est une option de compilation, pas du code. Et l'OpenMP est garanti bit-à-bit identique à la série. »

Message clé : Série, OpenMP, MPI, GPU : un drapeau cmake, opérateurs inchangés.

[Fin] Thank you for any feedback

Ce que je dis : « Pour résumer : une équation générique, deux contrats (physique locale, solveur elliptique), un coupleur qui les relie, et un backend choisi à la compilation. On obtient un solveur qui passe du laptop au GPU sans réécriture. Merci, je prends vos questions. »

Questions probables et réponses

Conception / C++

Pourquoi des concepts et pas de l'héritage virtuel ?

Performance et portabilité GPU. Une fonction virtuelle implique une vtable et un appel indirect : ça casse l'inlining dans les boucles chaudes, et ce n'est pas appelable proprement depuis un kernel device. Les concepts donnent un polymorphisme à la compilation : tout est inliné, zéro surcoût, et le même code part sur GPU. Le compilateur valide le contrat (`static_assert`) au lieu d'une hiérarchie de classes.

Le choix du backend à la compilation, ça ne manque pas de souplesse ?

C'est un choix assumé : on cible le HPC, où la machine est connue au build (un nœud GPU, un cluster MPI). Le gain est que les opérateurs restent monomorphes et pleinement optimisés. La flexibilité « runtime » qu'on veut vraiment (résolution, schéma au niveau utilisateur, modèle) reste accessible, notamment depuis Python qui pilote le binaire compilé sans recompiler.

Que veut dire `ADC_HD` exactement ?

C'est une macro : `__host__ __device__` quand on compile avec Kokkos/CUDA, vide sinon. Elle marque les fonctions appelables à la fois sur l'hôte et sur le GPU. Comme le modèle n'utilise que de l'arithmétique sur des petites vues (pas d'allocation, pas de `std::` non-device), la même fonction compile pour les deux cibles.

`StateVec<1>, n_vars` : c'est quoi ?

`State` est le vecteur d'état par cellule ; `StateVec<1>` = une variable (le diocotron transporte juste n_e). Pour Euler 2D ce serait `StateVec<4>` (densité, deux quantités de mouvement, énergie). `n_vars` expose cette taille au reste du code.

Physique

C'est quoi l'instabilité diocotron ?

L'instabilité d'une colonne de plasma non neutre (un faisceau d'électrons) en champs croisés $E \times B$. C'est une instabilité de cisaillement, l'analogie électrostatique de Kelvin–Helmholtz : le gradient de la vitesse de dérive déstabilise des perturbations azimutales, qui croissent exponentiellement. Le taux de croissance dépend du nombre de mode l .

Comment $\nabla\phi$ entre dans le flux ?

Le champ électrique est $E = -\nabla\phi$, et la vitesse de dérive est $v = (E \times B)/|B|^2$. Dans le code, $v_x = -\partial_y\phi/B_0$ et $v_y = +\partial_x\phi/B_0$. Le coupleur calcule $\nabla\phi$ par différences finies centrées sur la grille après chaque résolution de Poisson, et le range dans `aux`.

Le fond neutralisant n_{i0} / la moyenne $\langle\rho\rangle$: qui le calcule ?

Le terme elliptique est $\alpha(n_e - n_{i0})$: la densité moins un fond ionique. Pour un Poisson périodique, le second membre doit être de moyenne nulle (nullspace). La quantité globale ($\langle\rho\rangle$, le fond neutralisant) est calculée par le *coupleur*, pas par le modèle local : le modèle ne voit qu'une cellule, il ne peut pas connaître une moyenne globale. C'est un choix de conception : les quantités non locales remontent au coupleur.

Numérique

À quoi sert `max_wave_speed` ?

Au pas de temps. La condition CFL borne Δt par $CFL \cdot \Delta x / v_{\max}$; `max_wave_speed` fournit la vitesse caractéristique maximale locale, le coupleur en prend le max global pour fixer Δt .

van Leer, WENO5 : quand utiliser quoi ?

NoSlope (ordre 1) pour du robuste/diagnostic ; les limiteurs MUSCL (Minmod, van Leer, MC) pour de l'ordre 2 TVD sans osciller aux chocs ; WENO5-Z quand on veut de l'ordre élevé sur des structures lisses (taux de croissance, turbulence). Le choix est un paramètre template, donc on compare sans rien réécrire.

Comment Poisson est-il résolu ?

Trois backends. Multigrille géométrique (V-cycles, général, gère un bord conducteur embarqué) ; FFT spectral direct (périodique, exact, très rapide) ; FFT distribué pour le MPI. Tous remplissent le concept EllipticSolver, donc le coupleur est indifférent au choix.

Parallélisme / AMR / HPC

AMR : sous-cyclage et reflux, c'est conservatif ?

Oui. Le niveau fin fait plusieurs petits pas pour un pas grossier (sous-cyclage). Aux bords grossier/fin, les flux ne coïncident pas : un `FluxRegister` accumule la différence et corrige les cellules grossières (reflux). La somme utilise une accumulation déterministe (compensée), donc la masse est conservée à l'arrondi (dérive $\sim 10^{-13}$ à 10^{-15}) et le résultat est bit-à-bit reproductible.

Berger–Rigoutsos, c'est quoi ?

L'algorithme standard pour regrouper les cellules taguées en un petit nombre de patches rectangulaires efficaces. Il calcule des signatures (histogrammes des tags) et coupe là où le laplacien de la signature change de signe (un trou ou un coin). On évite ainsi un patch unique trop gros ou une myriade de petits.

Comment se fait la décomposition MPI ?

Un `BoxArray` pave le domaine en boîtes ; un `DistributionMapping` assigne chaque boîte à un rang ; un `MultiFab` est la collection distribuée des données. L'échange de halos (`fill_boundary`) passe par le `seam comm`. Le modèle et les schémas ne savent pas qu'on est en MPI : ils ne voient qu'une Box locale.

OpenMP / GPU bit-identique à la série, comment ?

Les réductions (max d'onde, masse) utilisent des réducteurs déterministes (somme/max à ordre fixe), pas une accumulation atomique dépendante de l'ordre des threads. On a vérifié : série et MPI donnent des sommes identiques bit-à-bit, et un run GPU (GH200 sur ROMEO) donne le même checksum que le CPU, avec `compute-sanitizer` à zéro erreur.

À quoi sert l'IMEX / l'Asymptotic Preserving dans TimeIntegrator ?

À traiter les termes raides implicitement. Quand le couplage devient raide (limite quasi-neutre, sources rapides), un schéma explicite imposerait un Δt minuscule. IMEX traite la partie raide en implicite, et un schéma AP reste consistant et stable dans la limite singulière. TimeIntegrator les compose sans connaître l'implémentation des opérateurs.

Validation / limites**Comment savez-vous que c'est correct ?**

Plusieurs niveaux. Taux de croissance diocotron comparés à l'analytique (modes 3/4/5, WENO5-Z + SSPRK3) : on retrouve les taux avec un léger sur-tir qui est *plat* en résolution. L'AMR suit l'uniforme à résolution effective égale, avec $\sim 40\%$ des cellules. Conservation de la masse à l'arrondi. Et la reproductibilité bit-à-bit série/MPI/GPU sert de test de non-régression fort.

Le sur-tir sur le taux de croissance, vous l'expliquez ?

C'est une question ouverte : le sur-tir dépend du mode l et reste structurel après avoir écarté plusieurs pistes (grille, observable, normalisation). La piste actuelle est le traitement des bords de l'anneau sur une grille cartésienne. C'est cohérent en résolution, donc ce n'est pas un défaut de convergence mais un biais de modélisation du bord.

En quoi c'est différent d'AMReX ou d'un framework existant ?

Ce n'est pas un concurrent généraliste. C'est une conception ciblée : les contrats (Physical-Model, EllipticSolver) et les seams sont volontairement minces, pour que le couplage hyperbolique-elliptique et le passage multi-backend soient lisibles et vérifiés. L'idée pédagogique : montrer qu'avec deux concepts et une couche de seams, on couvre série / OpenMP / MPI / GPU et l'AMR.

Qu'est-ce qui n'est pas encore fait ?

Honnêtement : le bord d'anneau cartésien (cf. sur-tir) ; l'extension à plus de modèles multi-variables couplés ; et un durcissement du multigrille près du bord conducteur embarqué à haute résolution, où le V-cycle peut diverger (documenté). Le socle architecture / parallélisme / AMR est en place et testé.